In [99]:
```
import pandas as pd
import warnings
warnings.filterwarnings("ignore")
```

Loading Dataset

In [100…
```
df=pd.read_csv(r'C:\Users\anupa\Desktop\Python Learning\Churn\WA_Fn-UseC_-Telco-
```

In [101…
```
df
```

Out[101…

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService
0	7590-VHVEG	Female	0	Yes	No	1	No
1	5575-GNVDE	Male	0	No	No	34	Yes
2	3668-QPYBK	Male	0	No	No	2	Yes
3	7795-CFOCW	Male	0	No	No	45	No
4	9237-HQITU	Female	0	No	No	2	Yes
...	...	...	...	...	...	...	...
7038	6840-RESVB	Male	0	Yes	Yes	24	Yes
7039	2234-XADUH	Female	0	Yes	Yes	72	Yes
7040	4801-JZAZL	Female	0	Yes	Yes	11	No
7041	8361-LTMKD	Male	1	Yes	No	4	Yes
7042	3186-AJIEK	Male	0	No	No	66	Yes

7043 rows × 21 columns

In [102…
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   customerID        7043 non-null   object
 1   gender            7043 non-null   object
 2   SeniorCitizen     7043 non-null   int64
 3   Partner           7043 non-null   object
 4   Dependents        7043 non-null   object
 5   tenure            7043 non-null   int64
 6   PhoneService      7043 non-null   object
 7   MultipleLines     7043 non-null   object
 8   InternetService   7043 non-null   object
 9   OnlineSecurity    7043 non-null   object
 10  OnlineBackup      7043 non-null   object
 11  DeviceProtection  7043 non-null   object
 12  TechSupport       7043 non-null   object
 13  StreamingTV       7043 non-null   object
 14  StreamingMovies   7043 non-null   object
 15  Contract          7043 non-null   object
 16  PaperlessBilling  7043 non-null   object
 17  PaymentMethod     7043 non-null   object
 18  MonthlyCharges    7043 non-null   float64
 19  TotalCharges      7043 non-null   object
 20  Churn             7043 non-null   object
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB
```

In [103…

```
pd.set_option('display.max_columns', None)
print(df)
df.head()
```

```
      customerID  gender  SeniorCitizen Partner Dependents  tenure  \
0     7590-VHVEG  Female              0     Yes         No       1
1     5575-GNVDE    Male              0      No         No      34
2     3668-QPYBK    Male              0      No         No       2
3     7795-CFOCW    Male              0      No         No      45
4     9237-HQITU  Female              0      No         No       2
...          ...     ...            ...     ...        ...     ...
7038  6840-RESVB    Male              0     Yes        Yes      24
7039  2234-XADUH  Female              0     Yes        Yes      72
7040  4801-JZAZL  Female              0     Yes        Yes      11
7041  8361-LTMKD    Male              1     Yes         No       4
7042  3186-AJIEK    Male              0      No         No      66

     PhoneService     MultipleLines InternetService OnlineSecurity  \
0              No  No phone service             DSL             No
1             Yes                No             DSL            Yes
2             Yes                No             DSL            Yes
3              No  No phone service             DSL            Yes
4             Yes                No     Fiber optic             No
...           ...               ...             ...            ...
7038          Yes               Yes             DSL            Yes
7039          Yes               Yes     Fiber optic             No
7040           No  No phone service             DSL            Yes
7041          Yes               Yes     Fiber optic             No
7042          Yes                No     Fiber optic            Yes

     OnlineBackup DeviceProtection TechSupport StreamingTV StreamingMovies  \
0             Yes               No          No          No              No
1              No              Yes          No          No              No
2             Yes               No          No          No              No
3              No              Yes         Yes          No              No
4              No               No          No          No              No
...           ...              ...         ...         ...             ...
7038           No              Yes         Yes         Yes             Yes
7039          Yes              Yes          No         Yes             Yes
7040           No               No          No          No              No
7041           No               No          No          No              No
7042           No              Yes         Yes         Yes             Yes

            Contract PaperlessBilling              PaymentMethod  \
0     Month-to-month              Yes           Electronic check
1           One year               No               Mailed check
2     Month-to-month              Yes               Mailed check
3           One year               No  Bank transfer (automatic)
4     Month-to-month              Yes           Electronic check
...              ...              ...                        ...
7038        One year              Yes               Mailed check
7039        One year              Yes    Credit card (automatic)
7040  Month-to-month              Yes           Electronic check
7041  Month-to-month              Yes               Mailed check
7042        Two year              Yes  Bank transfer (automatic)

      MonthlyCharges TotalCharges Churn
0              29.85        29.85    No
1              56.95       1889.5    No
2              53.85       108.15   Yes
3              42.30      1840.75    No
4              70.70       151.65   Yes
...              ...          ...   ...
7038           84.80       1990.5    No
```

```
7039          103.20       7362.9    No
7040           29.60       346.45    No
7041           74.40        306.6   Yes
7042          105.65       6844.5    No

[7043 rows x 21 columns]
```

Out[103…

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	Mul
0	7590-VHVEG	Female	0	Yes	No	1	No	
1	5575-GNVDE	Male	0	No	No	34	Yes	
2	3668-QPYBK	Male	0	No	No	2	Yes	
3	7795-CFOCW	Male	0	No	No	45	No	
4	9237-HQITU	Female	0	No	No	2	Yes	

In [104…

```
df_1_nums = df.select_dtypes(exclude='object')
df_1_nums
```

Out[104…

	SeniorCitizen	tenure	MonthlyCharges
0	0	1	29.85
1	0	34	56.95
2	0	2	53.85
3	0	45	42.30
4	0	2	70.70
...	...	...	...
7038	0	24	84.80
7039	0	72	103.20
7040	0	11	29.60
7041	1	4	74.40
7042	0	66	105.65

7043 rows × 3 columns

In [105…

```
df_1_objs = df.select_dtypes(include='object')
df_1_objs
```

Out[105…

	customerID	gender	Partner	Dependents	PhoneService	MultipleLines	InternetS
0	7590-VHVEG	Female	Yes	No	No	No phone service	
1	5575-GNVDE	Male	No	No	Yes	No	
2	3668-QPYBK	Male	No	No	Yes	No	
3	7795-CFOCW	Male	No	No	No	No phone service	
4	9237-HQITU	Female	No	No	Yes	No	Fib
...	...	...	...	...	...	...	
7038	6840-RESVB	Male	Yes	Yes	Yes	Yes	
7039	2234-XADUH	Female	Yes	Yes	Yes	Yes	Fib
7040	4801-JZAZL	Female	Yes	Yes	No	No phone service	
7041	8361-LTMKD	Male	Yes	No	Yes	Yes	Fib
7042	3186-AJIEK	Male	No	No	Yes	No	Fib

7043 rows × 18 columns

In [106…

```
df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce')
```

In [107…

```
df_1_objs = df.select_dtypes(include='object')
df_1_objs
```

Out[107…

	customerID	gender	Partner	Dependents	PhoneService	MultipleLines	InternetS
0	7590-VHVEG	Female	Yes	No	No	No phone service	
1	5575-GNVDE	Male	No	No	Yes	No	
2	3668-QPYBK	Male	No	No	Yes	No	
3	7795-CFOCW	Male	No	No	No	No phone service	
4	9237-HQITU	Female	No	No	Yes	No	Fib
...	...	...	...	...	...	...	
7038	6840-RESVB	Male	Yes	Yes	Yes	Yes	
7039	2234-XADUH	Female	Yes	Yes	Yes	Yes	Fib
7040	4801-JZAZL	Female	Yes	Yes	No	No phone service	
7041	8361-LTMKD	Male	Yes	No	Yes	Yes	Fib
7042	3186-AJIEK	Male	No	No	Yes	No	Fib

7043 rows × 17 columns

In [108…

```
df_1_nums = df.select_dtypes(exclude='object')
df_1_nums
```

Out[108…

	SeniorCitizen	tenure	MonthlyCharges	TotalCharges
0	0	1	29.85	29.85
1	0	34	56.95	1889.50
2	0	2	53.85	108.15
3	0	45	42.30	1840.75
4	0	2	70.70	151.65
...	...	...	...	...
7038	0	24	84.80	1990.50
7039	0	72	103.20	7362.90
7040	0	11	29.60	346.45
7041	1	4	74.40	306.60
7042	0	66	105.65	6844.50

7043 rows × 4 columns

In [109…

```
df["TotalCharges"] = df["TotalCharges"].astype(float)
```

In [110…

```
df.isnull().sum()
```

Out[110…

```
customerID           0
gender               0
SeniorCitizen        0
Partner              0
Dependents           0
tenure               0
PhoneService         0
MultipleLines        0
InternetService      0
OnlineSecurity       0
OnlineBackup         0
DeviceProtection     0
TechSupport          0
StreamingTV          0
StreamingMovies      0
Contract             0
PaperlessBilling     0
PaymentMethod        0
MonthlyCharges       0
TotalCharges        11
Churn                0
dtype: int64
```

In [111…

```
100* df.isnull().sum() / len(df)
```

Out[111…

```
customerID          0.000000
gender              0.000000
SeniorCitizen       0.000000
Partner             0.000000
Dependents          0.000000
tenure              0.000000
PhoneService        0.000000
MultipleLines       0.000000
InternetService     0.000000
OnlineSecurity      0.000000
OnlineBackup        0.000000
DeviceProtection    0.000000
TechSupport         0.000000
StreamingTV         0.000000
StreamingMovies     0.000000
Contract            0.000000
PaperlessBilling    0.000000
PaymentMethod       0.000000
MonthlyCharges      0.000000
TotalCharges        0.156183
Churn               0.000000
dtype: float64
```

In [112…

```
def percent_missing(df):
    percent_nan = 100* df.isnull().sum() / len(df)
    percent_nan = percent_nan[percent_nan>0].sort_values()
    return percent_nan
percent_nan = percent_missing(df)
percent_nan
```

Out[112…

```
TotalCharges    0.156183
dtype: float64
```

In [113…

```
# Handle missing values
df.dropna(inplace=True)
```

In [114…

```
100* df.isnull().sum() / len(df)
```

Out[114…

```
customerID          0.0
gender              0.0
SeniorCitizen       0.0
Partner             0.0
Dependents          0.0
tenure              0.0
PhoneService        0.0
MultipleLines       0.0
InternetService     0.0
OnlineSecurity      0.0
OnlineBackup        0.0
DeviceProtection    0.0
TechSupport         0.0
StreamingTV         0.0
StreamingMovies     0.0
Contract            0.0
PaperlessBilling    0.0
PaymentMethod       0.0
MonthlyCharges      0.0
TotalCharges        0.0
Churn               0.0
dtype: float64
```

In [115…

```
df_1=df.dropna(axis=0)
#df['TotalCharge'].fillna(value=df['TotalCharges'].mean())
# Handle missing values
#df['TotalCharges'].fillna(df['TotalCharges'].median(), inplace=True)
```

In [116…

```
df_1
```

Out[116…

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService
0	7590-VHVEG	Female	0	Yes	No	1	No
1	5575-GNVDE	Male	0	No	No	34	Yes
2	3668-QPYBK	Male	0	No	No	2	Yes
3	7795-CFOCW	Male	0	No	No	45	No
4	9237-HQITU	Female	0	No	No	2	Yes
...	...	...	...	...	...	...	...
7038	6840-RESVB	Male	0	Yes	Yes	24	Yes
7039	2234-XADUH	Female	0	Yes	Yes	72	Yes
7040	4801-JZAZL	Female	0	Yes	Yes	11	No
7041	8361-LTMKD	Male	1	Yes	No	4	Yes
7042	3186-AJIEK	Male	0	No	No	66	Yes

7032 rows × 21 columns

In [117…

```
100* df_1.isnull().sum() / len(df_1)
```

Out[117…

```
customerID          0.0
gender              0.0
SeniorCitizen       0.0
Partner             0.0
Dependents          0.0
tenure              0.0
PhoneService        0.0
MultipleLines       0.0
InternetService     0.0
OnlineSecurity      0.0
OnlineBackup        0.0
DeviceProtection    0.0
TechSupport         0.0
StreamingTV         0.0
StreamingMovies     0.0
Contract            0.0
PaperlessBilling    0.0
PaymentMethod       0.0
MonthlyCharges      0.0
TotalCharges        0.0
Churn               0.0
dtype: float64
```

In [118…

```
constant_cols = [c for c in df_1.columns if df_1[c].nunique() == 1]
constant_cols
```

Out[118…

```
[]
```

In [119…

```
df_1[constant_cols]
```

Out[119…

0
1
2
3
4
...
7038
7039
7040
7041
7042

7032 rows × 0 columns

In [120…

```
df_1['Churn'] = df_1['Churn'].map({'Yes': 1, 'No': 0})
```

In [121…

```
df_1["Churn"]
```

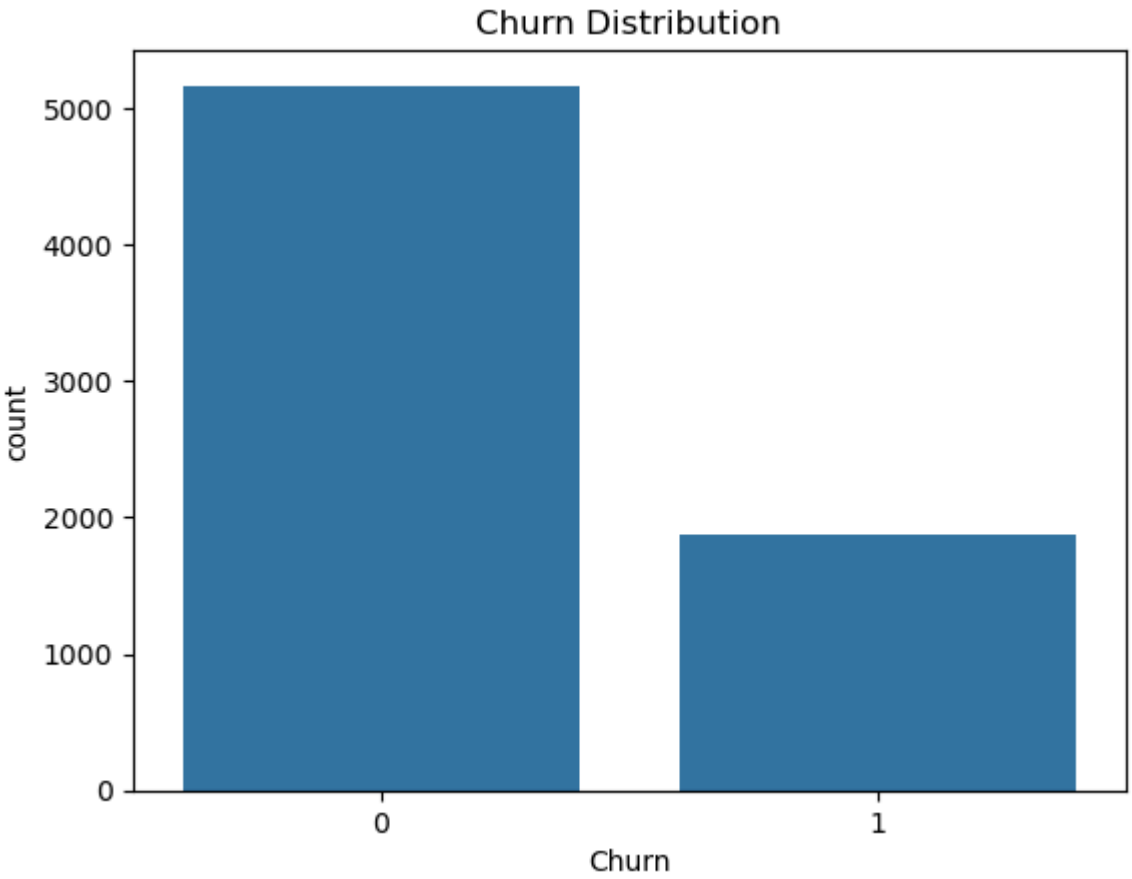
Out[121…

```
0       0
1       0
2       1
3       0
4       1
       ..
7038    0
7039    0
7040    0
7041    1
7042    0
Name: Churn, Length: 7032, dtype: int64
```

Churn Distribution

In [122…

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.countplot(x='Churn', data=df_1)
plt.title("Churn Distribution")
plt.show()
```

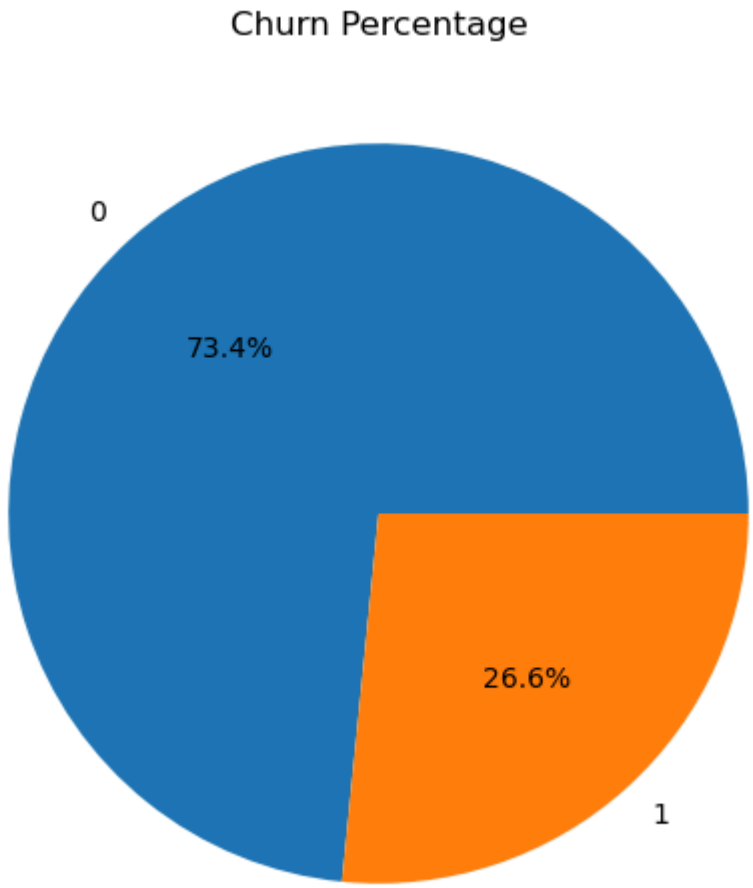


Churn Percentage Pie Chart

In [123…

```
df_1['Churn'].value_counts().plot.pie(
    autopct='%1.1f%%',
    figsize=(6,6),
    title="Churn Percentage"
```

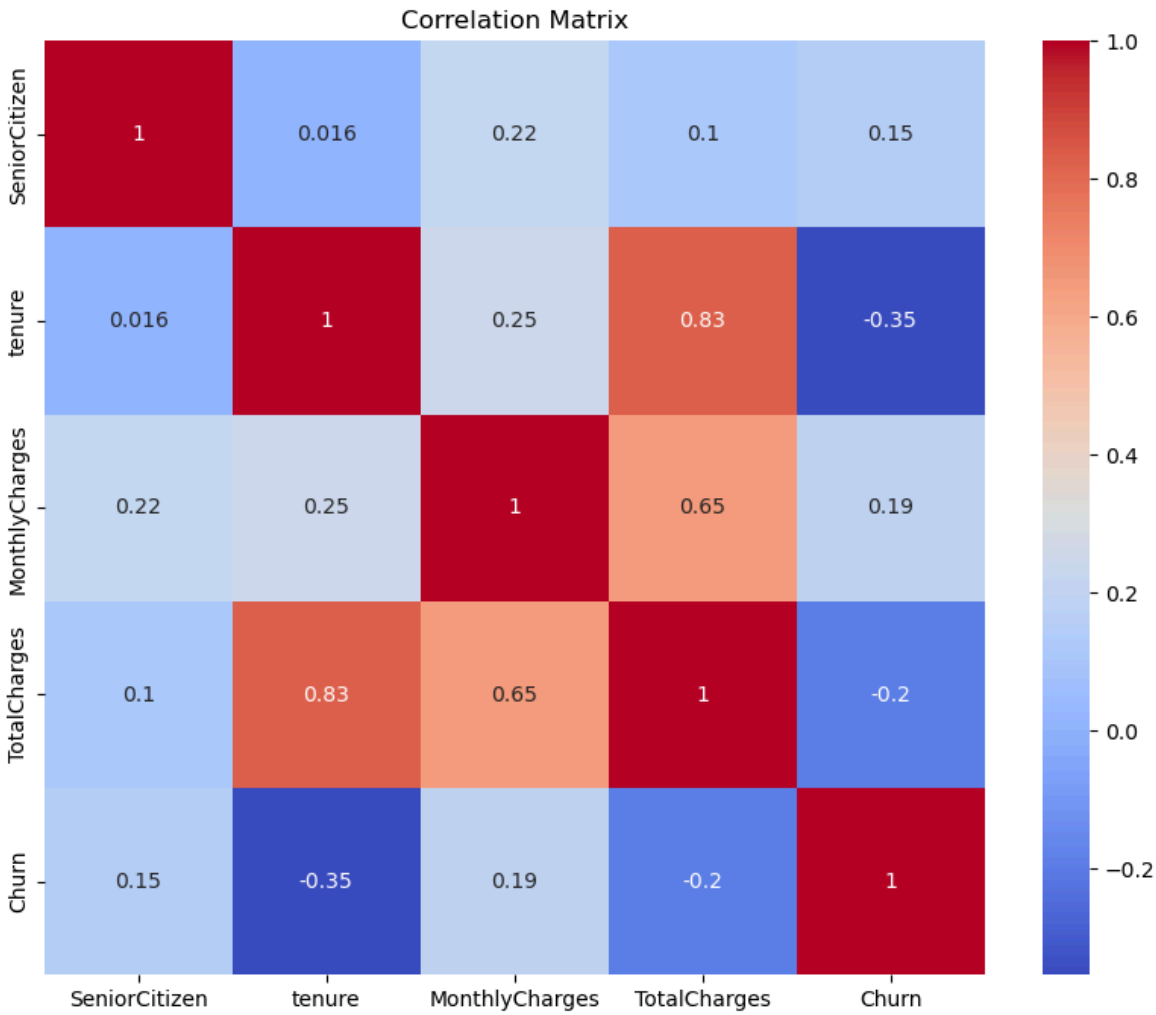
```
)
plt.ylabel("")
plt.show()
```



Correlation Heatmap

In [124…

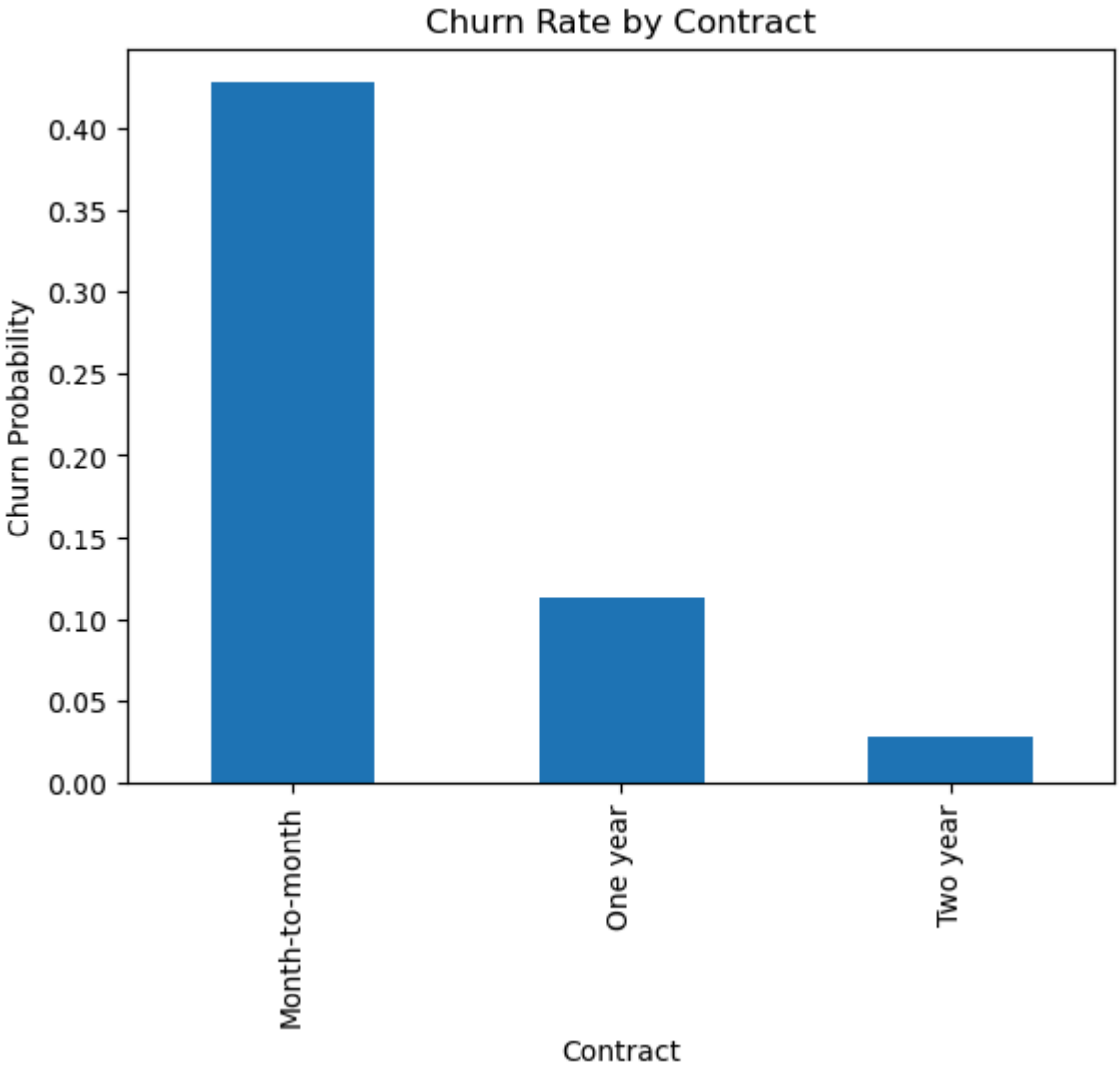
```
plt.figure(figsize=(10,8))
sns.heatmap(df_1.corr(numeric_only=True), annot=True, cmap="coolwarm")
plt.title("Correlation Matrix")
plt.show()
```



Churn Rate by Category

In [125…

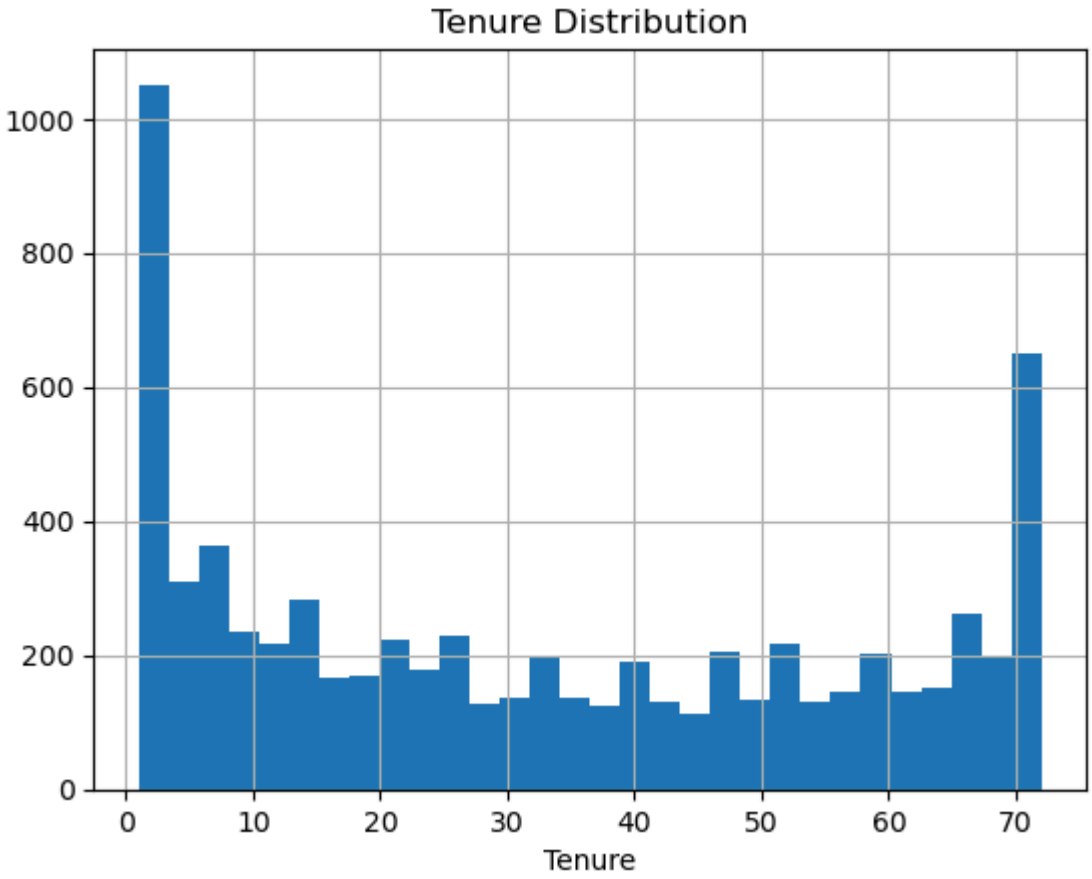
```
churn_rate = df_1.groupby('Contract')['Churn'].mean()

churn_rate.plot(kind='bar')
plt.title("Churn Rate by Contract")
plt.ylabel("Churn Probability")
plt.show()
```



Histogram

In [126…

```
df_1['tenure'].hist(bins=30)
plt.title("Tenure Distribution")
plt.xlabel("Tenure")
plt.show()
```



In [127…

```
# Create tenure group
df_1['tenure_group'] = pd.cut(df_1['tenure'],
                              bins=[0,12,24,48,72],
                              labels=['0-1yr','1-2yr','2-4yr','4-6yr'])
```

Categorical Variable Encoding

In [144…

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer

X = df_1.drop('Churn', axis=1)
y = df_1['Churn']

X = pd.get_dummies(X, drop_first=True)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42, stratify=y
)
```

Model- Logistic Regression

In [145…

```
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
```

```
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
```

Out[145…

```
▾ LogisticRegression
▶ Parameters
```

Evaluation Metrics

In [146…

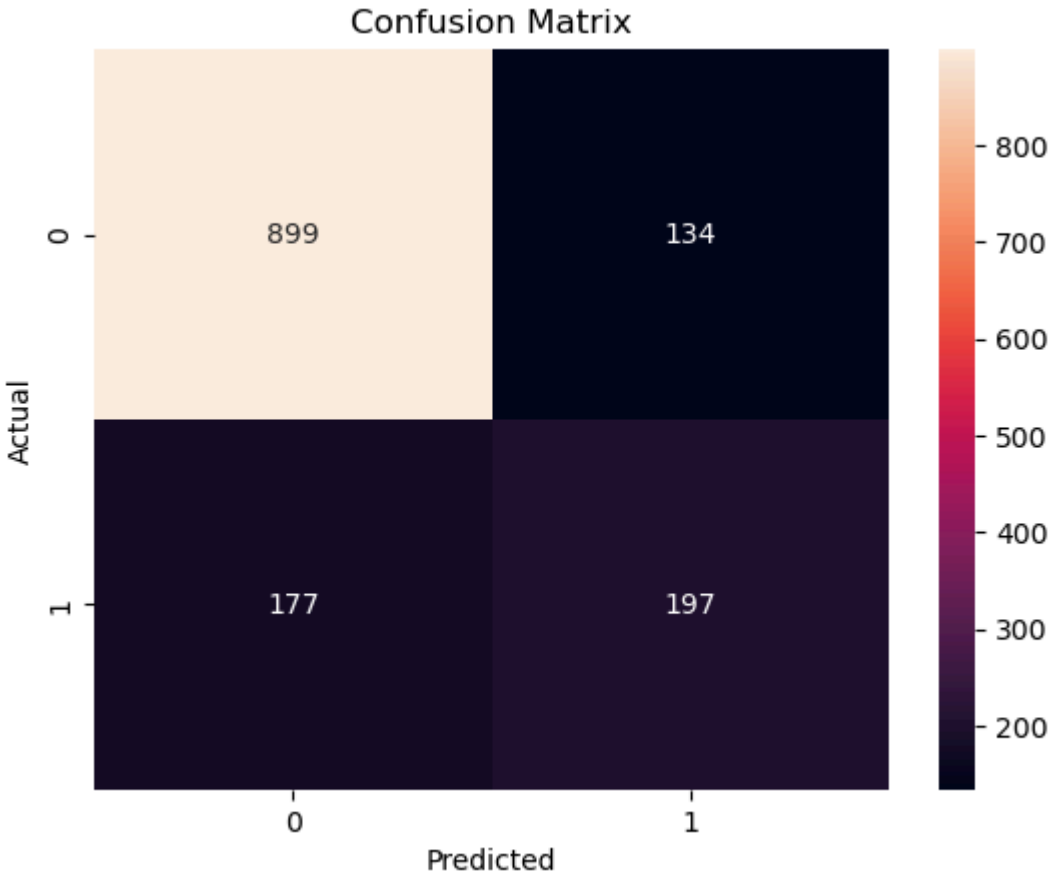
```
from sklearn.metrics import classification_report, roc_auc_score, confusion_matr
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.84      0.87      0.85      1033
           1       0.60      0.53      0.56       374

    accuracy                           0.78      1407
   macro avg       0.72      0.70      0.71      1407
weighted avg       0.77      0.78      0.77      1407
```

Confusion Matrix

In [147…

```
cm = confusion_matrix(y_test, y_pred)

sns.heatmap(cm, annot=True, fmt="d")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```



ROC-AUC Score

In [148…

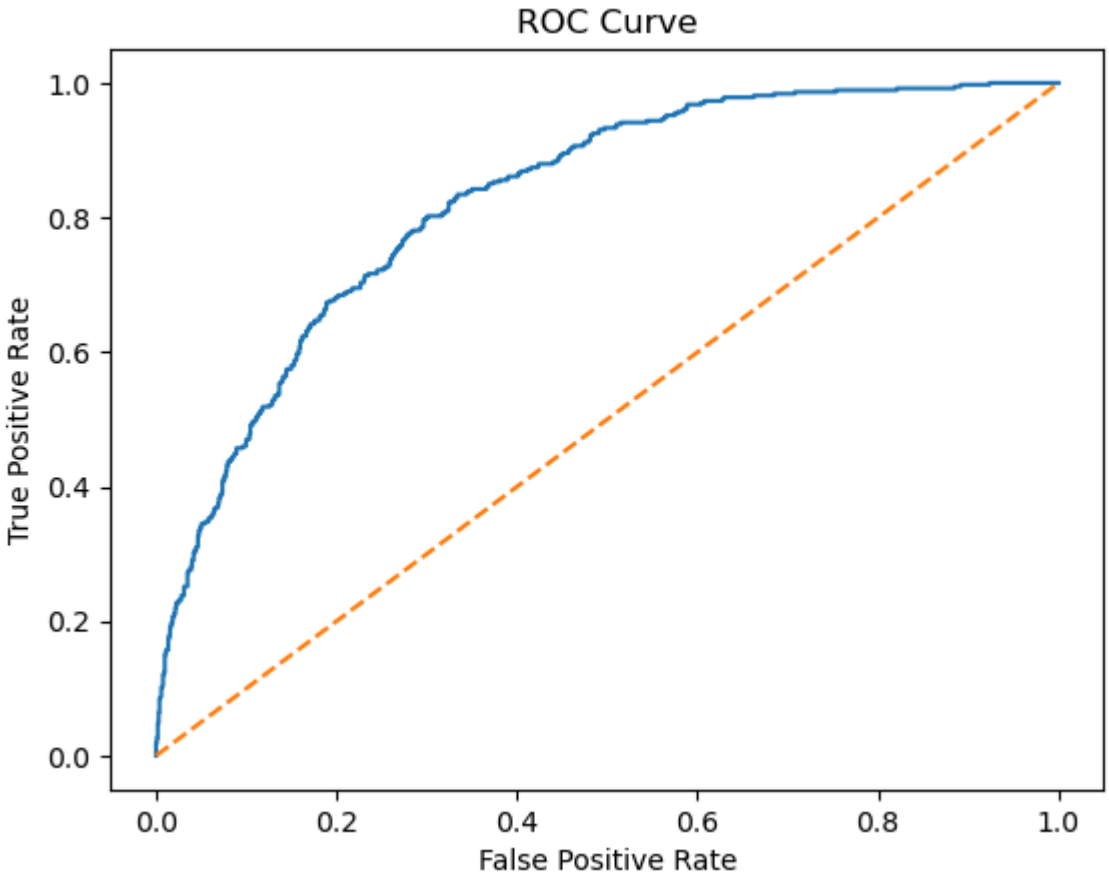
```
roc = roc_auc_score(y_test, y_prob)
gini=2*roc-1
print("ROC-AUC Score:", roc)
print("Gini:", gini)
```

```
ROC-AUC Score: 0.8256402358532077
Gini: 0.6512804717064153
```

ROC Curve

In [149…

```
from sklearn.metrics import roc_curve

fpr, tpr, thresholds = roc_curve(y_test, y_prob)

plt.plot(fpr, tpr)
plt.plot([0,1],[0,1],'--')

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")

plt.show()
```



Best Cutoff Using Youden Index

In [152…

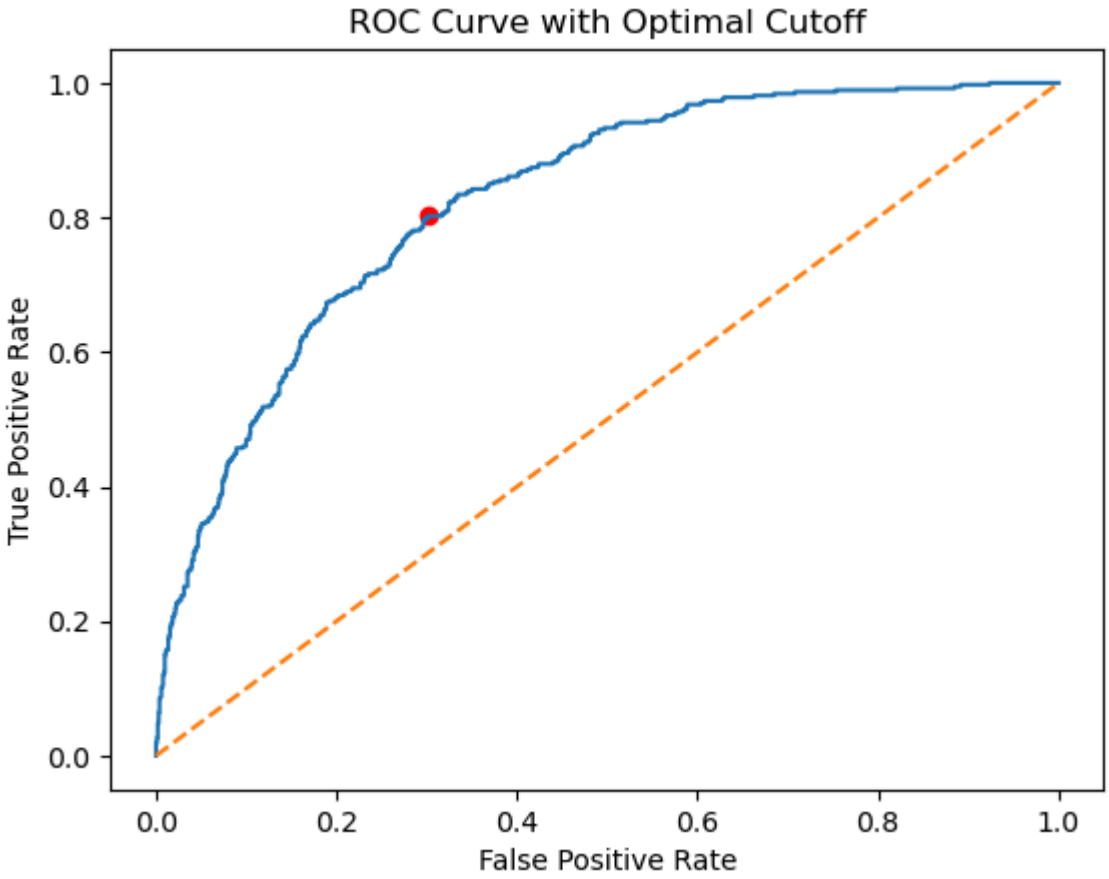
```
import numpy as np

youden_index = tpr - fpr

best_index = np.argmax(youden_index)

best_threshold = thresholds[best_index]

print("Best Threshold:", best_threshold)
```

```
Best Threshold: 0.23959133
```

Plot ROC Curve with Optimal Cutoff

In [154…

```
import matplotlib.pyplot as plt

plt.plot(fpr, tpr)
plt.plot([0,1],[0,1],'--')

plt.scatter(fpr[best_index], tpr[best_index], color='red')

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve with Optimal Cutoff")

plt.show()
```



In [156…

```
y_pred = model.predict(X_test)
y_pred
```

Out[156…

```
array([0, 1, 0, ..., 0, 0, 0], shape=(1407,))
```

In [157…

```
y_pred_new = (y_prob >= best_threshold).astype(int)
y_pred_new
```

Out[157…

```
array([0, 1, 0, ..., 0, 0, 0], shape=(1407,))
```

Evaluate with New Cutoff

In [158…

```
from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred_new))
```

```
              precision    recall  f1-score   support

           0       0.91      0.70      0.79      1033
           1       0.49      0.80      0.61       374

    accuracy                           0.73      1407
   macro avg       0.70      0.75      0.70      1407
weighted avg       0.80      0.73      0.74      1407
```

Another Method: Minimum Distance to Top Left

In [160…

```
distance = np.sqrt((1 - tpr)**2 + fpr**2)

best_index = np.argmin(distance)

best_threshold = thresholds[best_index]
best_threshold
```

Out[160…

```
np.float32(0.26976067)
```

Feature Importance (Logistic Coefficients)

In [153…

```
coefficients = pd.DataFrame({
    "Feature": X.columns,
    "Coefficient": model.coef_[0]
})

coefficients = coefficients.sort_values(by="Coefficient", ascending=False)

print(coefficients.head(10))
```

```
                            Feature  Coefficient
7041     InternetService_Fiber optic     0.490579
7059  PaymentMethod_Electronic check     0.391114
7057            PaperlessBilling_Yes     0.224272
2                     MonthlyCharges     0.209772
0                      SeniorCitizen     0.206237
411            customerID_0607-DAAHE     0.161686
4624           customerID_6532-YLWSI     0.161329
6047           customerID_8580-QVLOC     0.161232
6086           customerID_8631-NBHFZ     0.160989
4474           customerID_6323-AYBRX     0.160901
```

Model- Random Forest

In [166…

```
rf = RandomForestClassifier(random_state=42)
```

Hyperparameter Grid

In [167…

```
from sklearn.model_selection import RandomizedSearchCV

param_dist = {

    "n_estimators": [100, 200, 300, 400],

    "max_depth": [5, 8, 12, 15, None],

    "min_samples_split": [2, 5, 10],

    "min_samples_leaf": [1, 2, 4],

    "max_features": ["sqrt", "log2"]
}
```

Grid Search

In [170…

```
random_search = RandomizedSearchCV(

    estimator=rf,

    param_distributions=param_dist,

    n_iter=30,   # number of models tested

    cv=5,

    scoring="roc_auc",

    n_jobs=-1,

    random_state=42,

    verbose=1
)

random_search.fit(X_train, y_train)
```

```
Fitting 5 folds for each of 30 candidates, totalling 150 fits
```

Out[170…

```
▸ RandomizedSearchCV

▸ best_estimator_: RandomForestClassifier

  ▸ RandomForestClassifier
```

Best Parameters

In [171…

```
print(random_search.best_params_)
```

```
{'n_estimators': 200, 'min_samples_split': 10, 'min_samples_leaf': 1, 'max_featur
es': 'sqrt', 'max_depth': None}
```

Best Model

In [173…

```
best_rf = random_search.best_estimator_
best_rf
```

Out[173…

```
▾ RandomForestClassifier

▶ Parameters
```

Predictions

In [175…
```
y_pred = best_rf.predict(X_test)
y_prob = best_rf.predict_proba(X_test)[:,1]
```

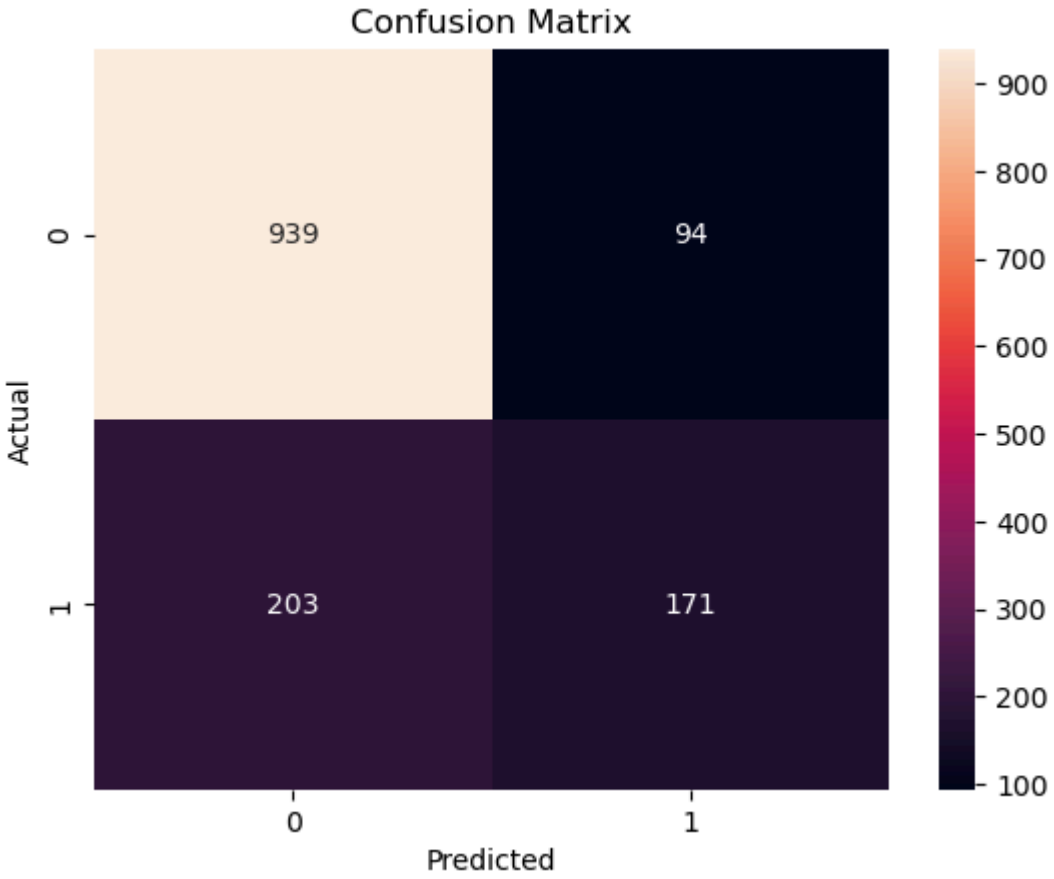
Model Evaluation

In [176…
```
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.82      0.91      0.86      1033
           1       0.65      0.46      0.54       374

    accuracy                           0.79      1407
   macro avg       0.73      0.68      0.70      1407
weighted avg       0.78      0.79      0.78      1407
```

Confusion Matrix

In [177…
```
cm = confusion_matrix(y_test, y_pred)

sns.heatmap(cm, annot=True, fmt="d")

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")

plt.show()
```



ROC-AUC

In [179…

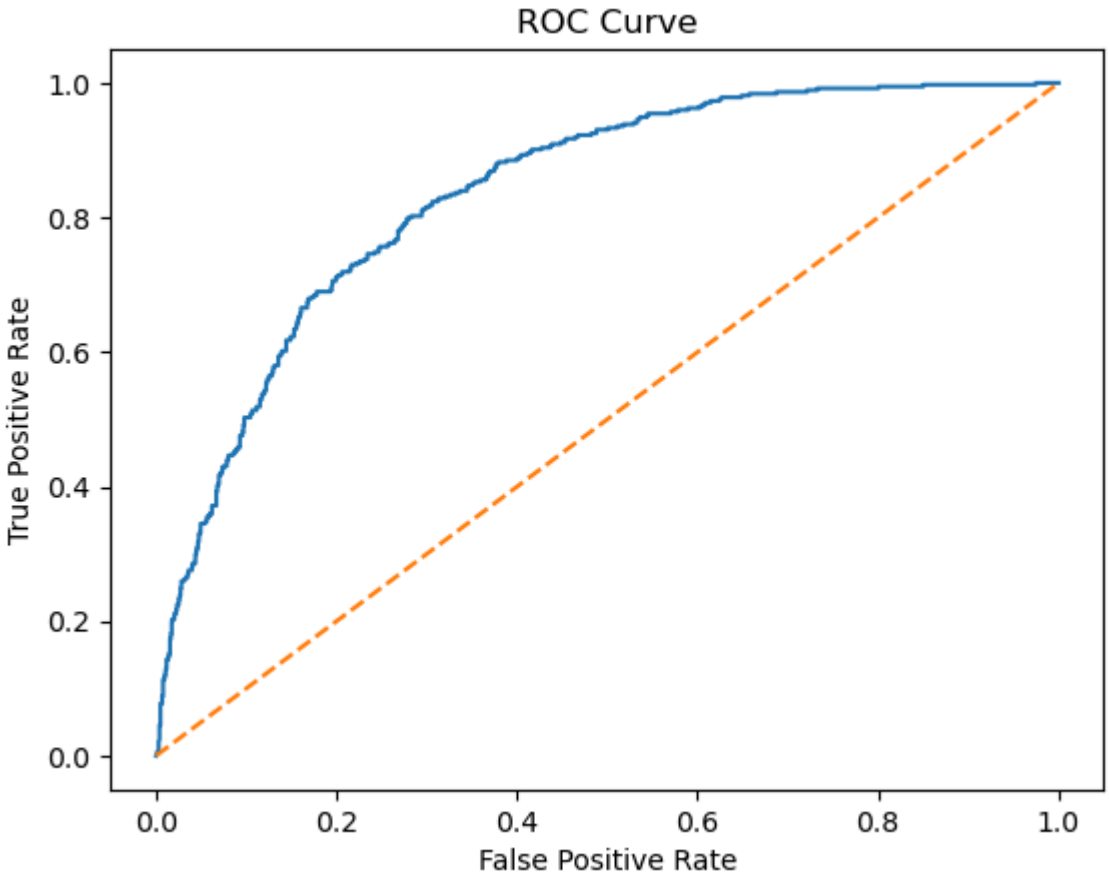
```
roc = roc_auc_score(y_test, y_prob)
gini=2*roc-1
print("ROC-AUC Score:", roc)
print("Gini:", gini)
```

```
ROC-AUC Score: 0.8363147159770359
Gini: 0.6726294319540718
```

ROC Curve

In [180…

```
fpr, tpr, thresholds = roc_curve(y_test, y_prob)

plt.plot(fpr, tpr)
plt.plot([0,1],[0,1],'--')

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")

plt.title("ROC Curve")

plt.show()
```



Find Optimal Cutoff

In [181…

```
youden_index = tpr - fpr

best_index = np.argmax(youden_index)

best_threshold = thresholds[best_index]

print("Best Threshold:", best_threshold)
```

```
Best Threshold: 0.25598971739045273
```

Apply New Cutoff

In [182…

```
y_pred_new = (y_prob >= best_threshold).astype(int)

print(classification_report(y_test, y_pred_new))
```

```
              precision    recall  f1-score   support

           0       0.91      0.72      0.80      1033
           1       0.51      0.80      0.62       374

    accuracy                           0.74      1407
   macro avg       0.71      0.76      0.71      1407
weighted avg       0.80      0.74      0.76      1407
```

Feature Importance

In [183…

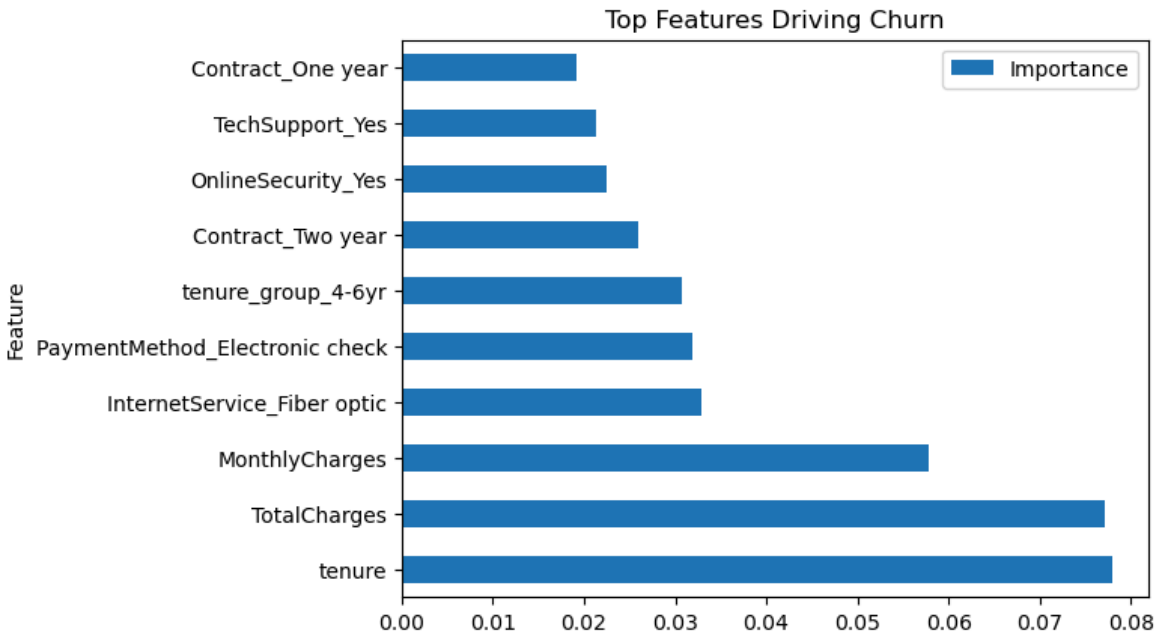
```
importance = pd.DataFrame({

    "Feature": X.columns,

    "Importance": best_rf.feature_importances_
})

importance = importance.sort_values(by="Importance", ascending=False)

print(importance.head(10))
```

```
                            Feature  Importance
1                            tenure    0.078018
3                      TotalCharges    0.077066
2                    MonthlyCharges    0.057893
7041     InternetService_Fiber optic    0.032894
7059  PaymentMethod_Electronic check    0.031866
7063              tenure_group_4-6yr    0.030686
7056               Contract_Two year    0.025869
7044              OnlineSecurity_Yes    0.022550
7050                 TechSupport_Yes    0.021284
7055               Contract_One year    0.019197
```

Feature Importance Plot

In [184…

```
importance.head(10).plot(

    x="Feature",

    y="Importance",

    kind="barh"
)

plt.title("Top Features Driving Churn")

plt.show()
```



Model- XGBOOST

In [185…

```
xgb = XGBClassifier(
    objective="binary:logistic",
    eval_metric="logloss",
    random_state=42
)
```

Hyperparameter Space

In [186…

```
param_dist = {

    "n_estimators": [100, 200, 300],

    "max_depth": [3, 5, 7],

    "learning_rate": [0.01, 0.05, 0.1],

    "subsample": [0.7, 0.8, 1],

    "colsample_bytree": [0.7, 0.8, 1],

    "gamma": [0, 1, 5]
}
```

Randomized Search

In [187…

```
random_search = RandomizedSearchCV(

    estimator=xgb,

    param_distributions=param_dist,
```

```
    n_iter=30,

    cv=5,

    scoring="roc_auc",

    n_jobs=-1,

    random_state=42,

    verbose=1
)

random_search.fit(X_train, y_train)
```

```
Fitting 5 folds for each of 30 candidates, totalling 150 fits
```

Out[187…

```
▸ RandomizedSearchCV

▸ best_estimator_:
  XGBClassifier

  ▸ XGBClassifier
```

Best Parameters

In [188…

```
print(random_search.best_params_)
```

```
{'subsample': 0.7, 'n_estimators': 300, 'max_depth': 5, 'learning_rate': 0.05, 'g
amma': 5, 'colsample_bytree': 0.7}
```

Best Model

In [189…

```
best_xgb = random_search.best_estimator_
```

Predictions

In [191…

```
y_pred = best_xgb.predict(X_test)

y_prob = best_xgb.predict_proba(X_test)[:,1]
```

Model Evaluation

In [192…

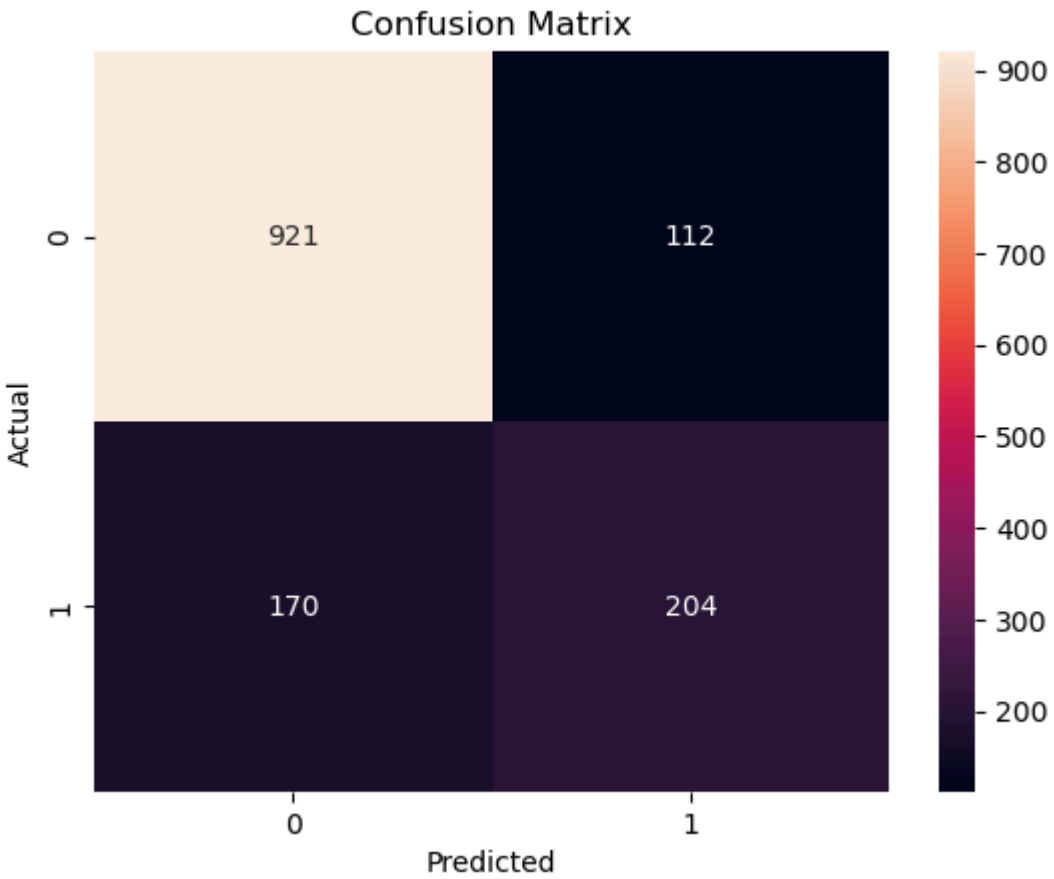
```
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.84      0.89      0.87      1033
           1       0.65      0.55      0.59       374

    accuracy                           0.80      1407
   macro avg       0.74      0.72      0.73      1407
weighted avg       0.79      0.80      0.79      1407
```

Confusion Matrix

In [193…

```
cm = confusion_matrix(y_test, y_pred)

sns.heatmap(cm, annot=True, fmt="d")

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")

plt.show()
```



ROC-AUC Score

In [195…

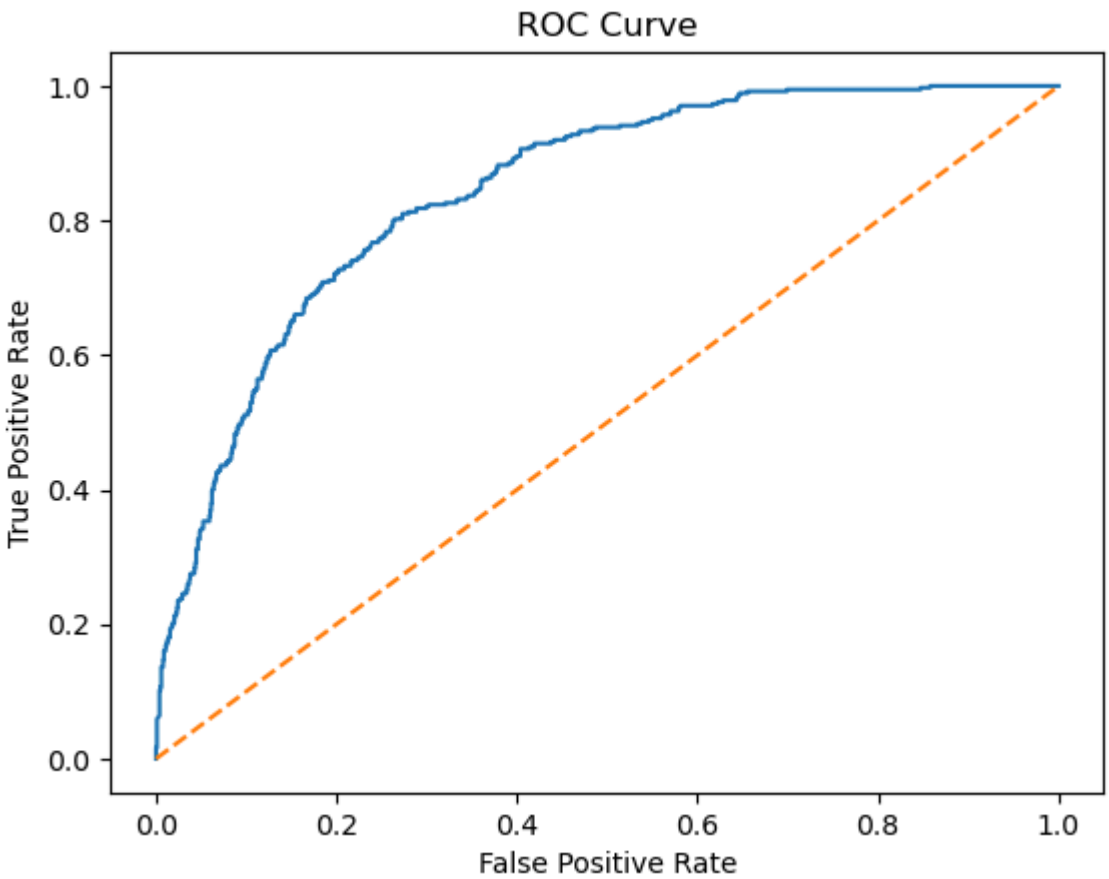
```
roc = roc_auc_score(y_test, y_prob)
Gini=2*roc-1
print("ROC-AUC Score:", roc)
print("Gini:", Gini)
```

```
ROC-AUC Score: 0.8434508803081208
Gini: 0.6869017606162415
```

ROC Curve

In [196…

```
fpr, tpr, thresholds = roc_curve(y_test, y_prob)

plt.plot(fpr, tpr)
plt.plot([0,1],[0,1],'--')

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")

plt.title("ROC Curve")

plt.show()
```



Find Optimal Cutoff

In [197…

```
youden_index = tpr - fpr

best_index = np.argmax(youden_index)

best_threshold = thresholds[best_index]

print("Best Threshold:", best_threshold)
```

```
Best Threshold: 0.28821978
```

Apply New Cutoff

In [198…

```
y_pred_new = (y_prob >= best_threshold).astype(int)

print(classification_report(y_test, y_pred_new))
```

```
              precision    recall  f1-score   support

           0       0.91      0.74      0.81      1033
           1       0.52      0.80      0.63       374

    accuracy                           0.75      1407
   macro avg       0.72      0.77      0.72      1407
weighted avg       0.81      0.75      0.77      1407
```

Feature Importance

In [199…

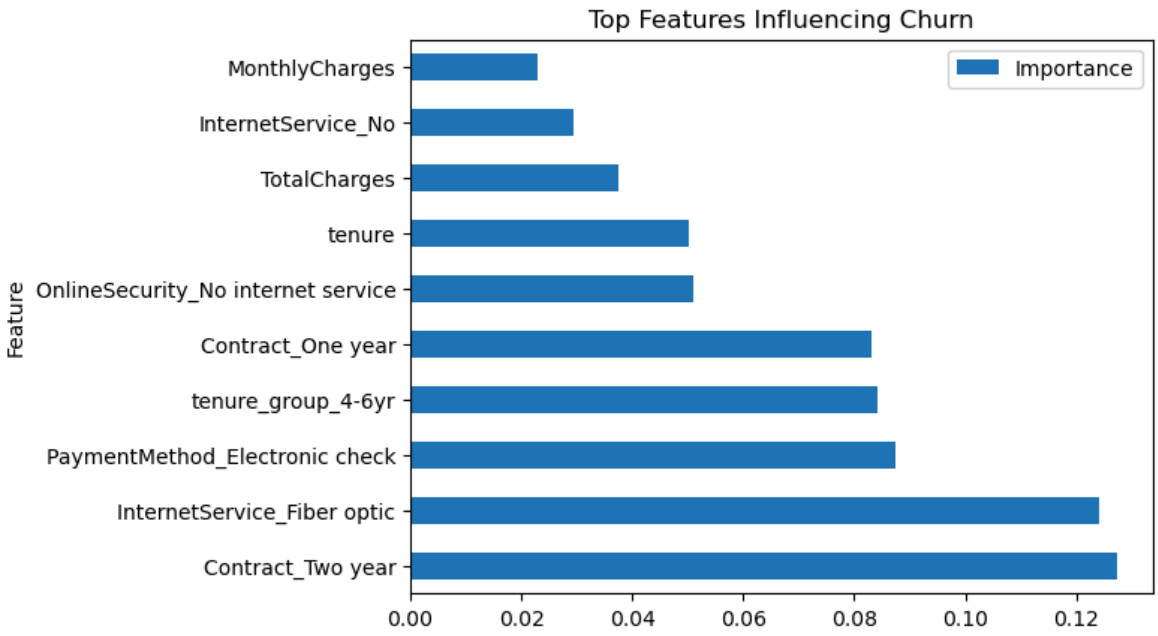
```
importance = pd.DataFrame({

    "Feature": X.columns,

    "Importance": best_xgb.feature_importances_
})

importance = importance.sort_values(by="Importance", ascending=False)

print(importance.head(10))
```

```
                                Feature  Importance
7056                   Contract_Two year    0.127474
7041         InternetService_Fiber optic    0.124170
7059      PaymentMethod_Electronic check    0.087358
7063                  tenure_group_4-6yr    0.084302
7055                   Contract_One year    0.083213
7043  OnlineSecurity_No internet service    0.051067
1                                 tenure    0.050097
3                           TotalCharges    0.037435
7042                  InternetService_No    0.029344
2                         MonthlyCharges    0.022796
```

Feature Importance Plot

In [200…

```
importance.head(10).plot(
    x="Feature",
    y="Importance",
    kind="barh"
)

plt.title("Top Features Influencing Churn")

plt.show()
```


Top Features Influencing Churn
Importance
MonthlyCharges
InternetService_No
TotalCharges
tenure
OnlineSecurity_No internet service
Contract_One year
tenure_group_4-6yr
PaymentMethod_Electronic check
InternetService_Fiber optic
Contract_Two year
Feature
0.00
0.02
0.04
0.06
0.08
0.10
0.12